UNIVERSIDADE FEDERAL DO RIO DE JANEIRO

CENTRO DE TECNOLOGIA

ESCOLA POLITÉCNICA

Relatório: Unidade Lógica e Aritmética (ULA)

Sistemas Digitais

2025.1

Alunos:

Dimitri (- - - - - - - -),
Felipe Vasconcellos (119154566),
Gustavo Moura (119156071).

Rio de Janeiro

1. Introdução

A Unidade Lógica e Aritmética (ULA) é responsável pela execução de operações lógicas e aritméticas em processadores e outros dispositivos computacionais. Neste contexto, o presente projeto consiste na implementação de uma ULA de 4 bits com capacidade de realizar 8 operações distintas, algumas lógicas (AND, OR, XOR e deslocador) e outras aritméticas (somador, subtrator, complemento de 2 e adição de 1).

Este trabalho inclui uma descrição detalhada do projeto com ênfase nos seguintes tópicos:

- **Diagrama de estados e diagrama de blocos:** ilustrando a estrutura e o fluxo de operações da ULA;

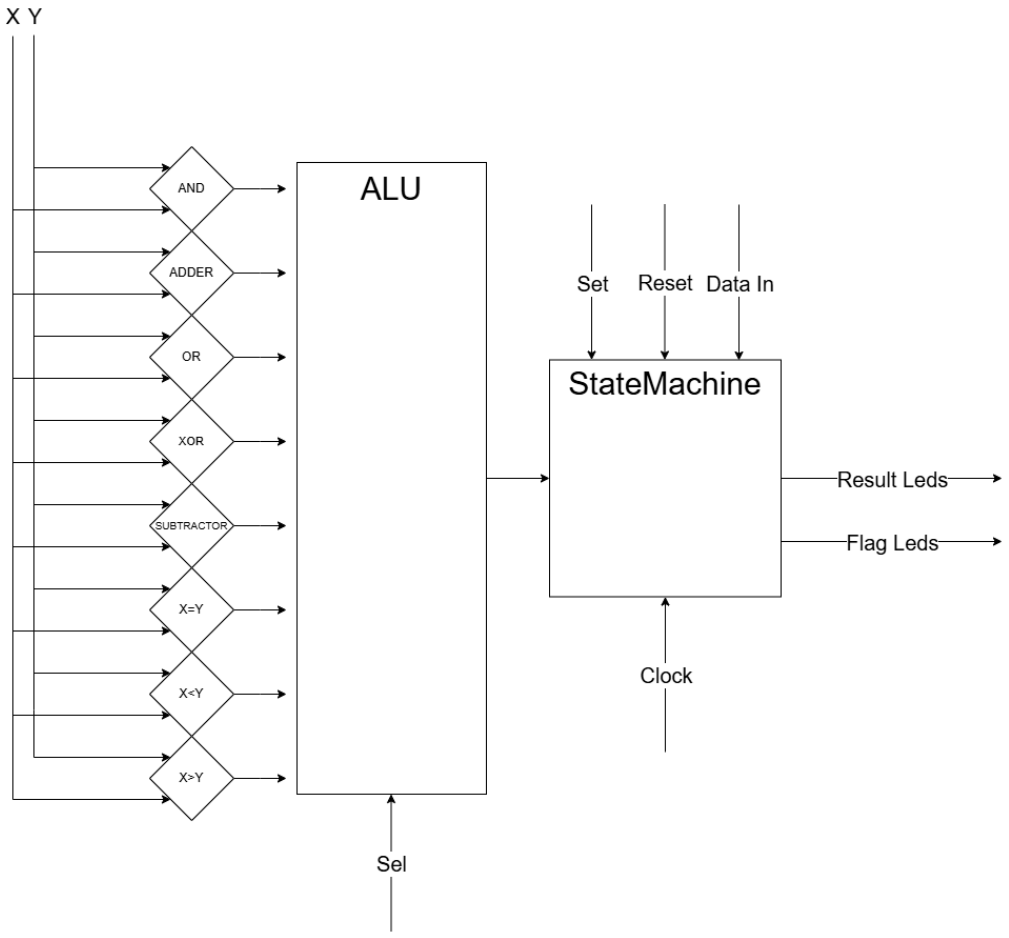
- **Descrição formal de cada bloco funcional:** explicando sua contribuição para o funcionamento do circuito;

- **Código VHDL comentado:** implementando a ULA de forma modular e eficiente;

- **Simulações pertinentes:** validando o comportamento do circuito em diferentes cenários;

- **Problema do bouncing:** um desafio comum em sistemas digitais que pode causar leituras incorretas de sinais e, portanto, serão discutidas as medidas adotadas para mitigar este efeito, garantindo a confiabilidade e entrada dos dados.

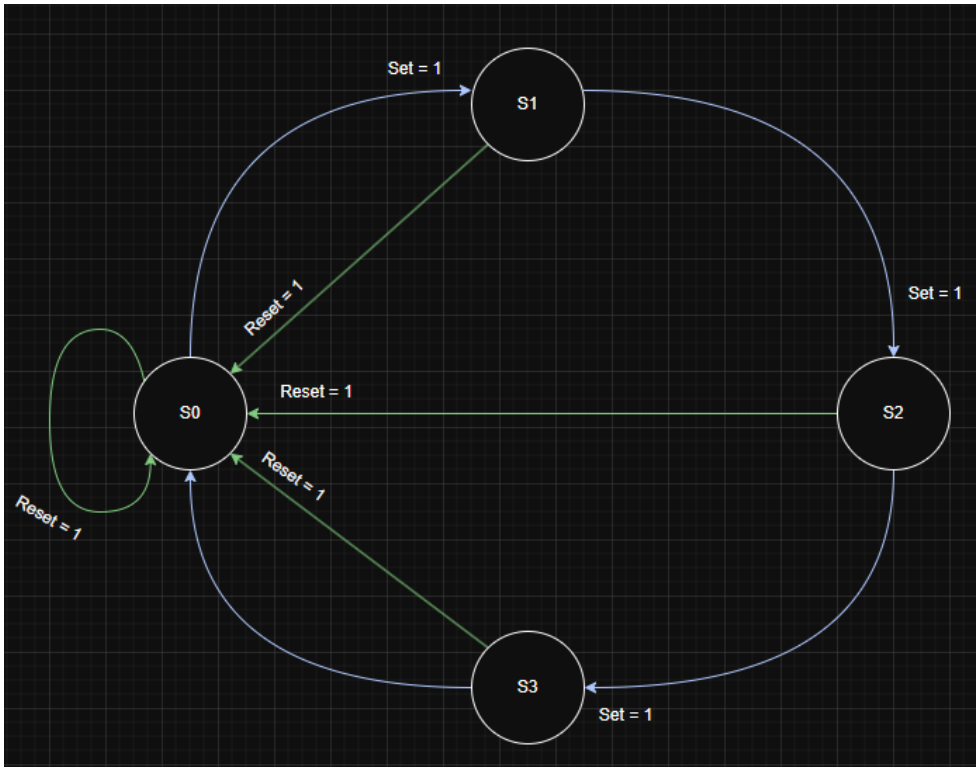
2. Descrição formal de cada bloco utilizado



- **StateMachine (bloco top level)**

Através de 4 processos, este bloco controla a captura de entradas e exibição de saídas. Há dois processos que cuidam do debounce dos botões set e reset. Logo em seguida, há um processo que cuida da atualização dos estados. Depois há um processo que captura os valores de entrada (numOp, num1 e num2) e, por fim, um processo que exibe as saídas.

Estado Atual	resetBtn	setBtn	Próximo Estado	Saída
S0	1	X	S0	0001
S0	0	0	S0	0001
S0	0	1	S1	0001
S1	1	X	S0	0001
S1	0	0	S1	0010
S1	0	1	S2	0010
S2	1	X	S0	0001
S2	0	0	S2	0011
S2	0	1	S3	0011
S3	1	X	S0	0001
S3	0	0	S3	resultado
S3	0	1	S3	resultado



- **ALU**

É o bloco da unidade lógico-aritmética. Este bloco utiliza os blocos: AND, Adder, OR, XOR, Subtrator, X = Y, X < Y e X > Y e uma condicional que seleciona as saídas. Tem a mesma funcionalidade de um mux 8:1.

- ## Bloco FullAdder

Descrição:

O bloco somador de 1 bit (Full Adder) realiza a soma de dois bits individuais (A e B) e um carry-in (Cin), gerando um bit de resultado (S) e um carry-out (Cout).

Funcionamento:

- Utiliza portas XOR para calcular a soma dos bits.
- Equações:
  - $S = A \oplus B \oplus Cin$
  - Cout = (A AND B) OR (Cin AND (A XOR B))

- ## Bloco FourBitAdder

Descrição:

Somador de 4 bits que combina quatro Full Adders (FA_0 a FA_3) para somar dois números binários de 4 bits (A[3:0] e B[3:0]).

Funcionamento:

Cada Full Adder soma um par de bits (A[i] + B[i]) e propaga o carry-out para o próximo estágio. O carry-in do FA_0 é 0 (para somas simples). Saída: Resultado de 4 bits (S[3:0]) e carry-out final (Cout).

- ## Bloco Subtrator de 4 Bits

Descrição:

Realiza a subtração de dois números de 4 bits (A − B) usando complemento de 2.

Funcionamento:

Converte B para seu complemento de 2 usando o bloco twosComplementer.
Soma A com o resultado do complemento usando o fourBitAdder.
Exemplo:

- A = 0101 (5), B = 0011 (3) → Complemento de B = 1101 (−3 em complemento de 2)
- Soma: 0101 + 1101 = 0010 (2, com carry-out ignorado).

Blocos Lógicos Bit a Bit

Descrição:

andGate:

- Operação: Saída = A AND B (para cada bit).
- Uso: Máscaras de bits ou operações lógicas.

xorGate:

- Operação: Saída = A XOR B (bit a bit).
- Uso: Comparação de bits ou geração de paridade.

orGate:

- Operação: Saída = A OR B (bit a bit).
- Uso: Combinação de flags ou fusão de dados.

Bloco Comparador de Igualdade

Descrição:

Compara dois números de 4 bits cada e dizem se são iguais.

Funcionamento:

Usam lógica combinacional através de portas lógicas xnor e and.
Exemplo:

- A = 0101 (5), B = 0101 (5)
- Saída X=Y: 0001.

Bloco Comparador de Superioridade

Descrição:

Compara dois números de 4 bits cada e dizem se X > Y.

Funcionamento:

Usam lógica combinacional através de portas lógicas and e or.
Exemplo:

- A = 0101 (5), B = 0011 (3)
- Saída X>Y: 0001.

Bloco Comparador de Inferioridade

Descrição:

Compara dois números de 4 bits cada e dizem se X < Y.

Funcionamento:

Usam lógica combinacional através de portas lógicas and e or.
Exemplo:

- A = 0001 (1), B = 0011 (3)
- Saída X<Y: 0001.

Bloco AddOne

Descrição:

Incrementa um número de 4 bits em 1.

Funcionamento:

Soma binária simples: Entrada + 0001.

Exemplo:

- Entrada: 1011 (11) → Saída: 1100 (12).

Bloco TwosComplementer

Descrição:

Calcula o complemento de 2 de um número de 4 bits (para representação de negativos).

Funcionamento:

Inverte todos os bits (NOT).
Adiciona 1 usando o bloco addOne.

- **Exemplo:**
    - Entrada: 0011 (3) → NOT = 1100 → +1 = 1101 (−3 em complemento de 2).

Lembrando que Todos os blocos são projetados para operações em binário de 4 bits, exceto o FullAdder (1 bit). O uso de complemento de 2 permite a representação de números negativos e simplifica operações aritméticas.

3. Problema do bouncing

3.1 Tratamento de debounce do botão *set*

Quando um botão físico é pressionado (em um contexto eletrônico), o contato mecânico que ocorre no botão não se encerra de forma instantânea e limpa. Na prática, ele gera uma série de pequenos contatos e desconexões muito rápidas antes de estabilizar tanto no estado pressionado quanto liberado, este fenômeno é conhecido como “bounces”.

Cabe destacar que esses "bounces" acontecem em poucos milissegundos, mas para um microcontrolador ou circuito digital, que pode ler o estado do botão a cada microssegundo, isso pode ser interpretado como múltiplos pressionamentos rápidos, podendo prejudicar o desenvolvimento de qualquer projeto neste cenário

Desta forma, neste projeto da ULA, o debounce foi implementado para cumprir a sincronização do sinal do botão ao clock do sistema, eliminando ruídos e metaestabilidade e, também, a geração de um pulso único em ‘debouncedSet’ para cada pressão do botão para contribuir com a confiabilidade da detecção do evento.

3.2 Sincronização do sinal do botão

O sinal de entrada ‘setBtn’ é proveniente de um botão físico sendo, portanto, suscetível a ruídos e transições instáveis no momento de seu acionamento. Para garantir a integridade e adaptação desse sinal ao domínio de clock do sistema, foi implementada a técnica de dupla sincronização.

O sinal ‘setBtn’ se manifesta por dois flip-flops em série (‘setBtnSync1’ e ‘setBtnSync2’). Esse procedimento minimiza os riscos de metaestabilidade e garante que o ‘setBtnSync2’ possa representar uma versão estável e sincronizada do botão, livre de oscilações rápidas decorrentes de ruídos mecânicos.

3.3 Detecção de borda de subida

Após a sincronização, a lógica do circuito identifica transições de borda de subida (0 → 1) no sinal ‘setBtnSync2’. Essa detecção é realizada mediante a comparação entre o valor atual de ‘setBtnSync2’ e o valor previamente armazenado em ‘setBtnPrev’.

Quando ocorre a condição setBtnSync2 = '1' e setBtnPrev = '0', o sistema reconhece que o botão foi pressionado. Nesse momento, o sinal ‘debouncedSet’ é ativado, gerando um pulso único de duração de um ciclo de clock. Esse pulso indica com precisão o acionamento do botão.

Posteriormente, setBtnPrev é atualizado com o valor de setBtnSync2, habilitando a detecção de futuras transições.

4. Conclusão

Após a implementação do código do projeto em VHDL, conseguimos verificar as operações funcionando de forma correta na simulação dentro do software ISE. No entanto, ao tentarmos executar diretamente na placa Xilinx Spartan 3AN-Starter Kit verificamos que a mudança de estados não ocorreu.